

Smartphone BI Agent

Complete Technical Documentation

An end-to-end Business-Intelligence system that predicts the market success of a new smartphone — segment, demand, customer profile, product-market fit, optimal price and live market intelligence — from launch-time specifications alone.

Architecture · Machine-Learning Models · Data · Backend · Frontend · Mathematics

Generated for academic / thesis documentation

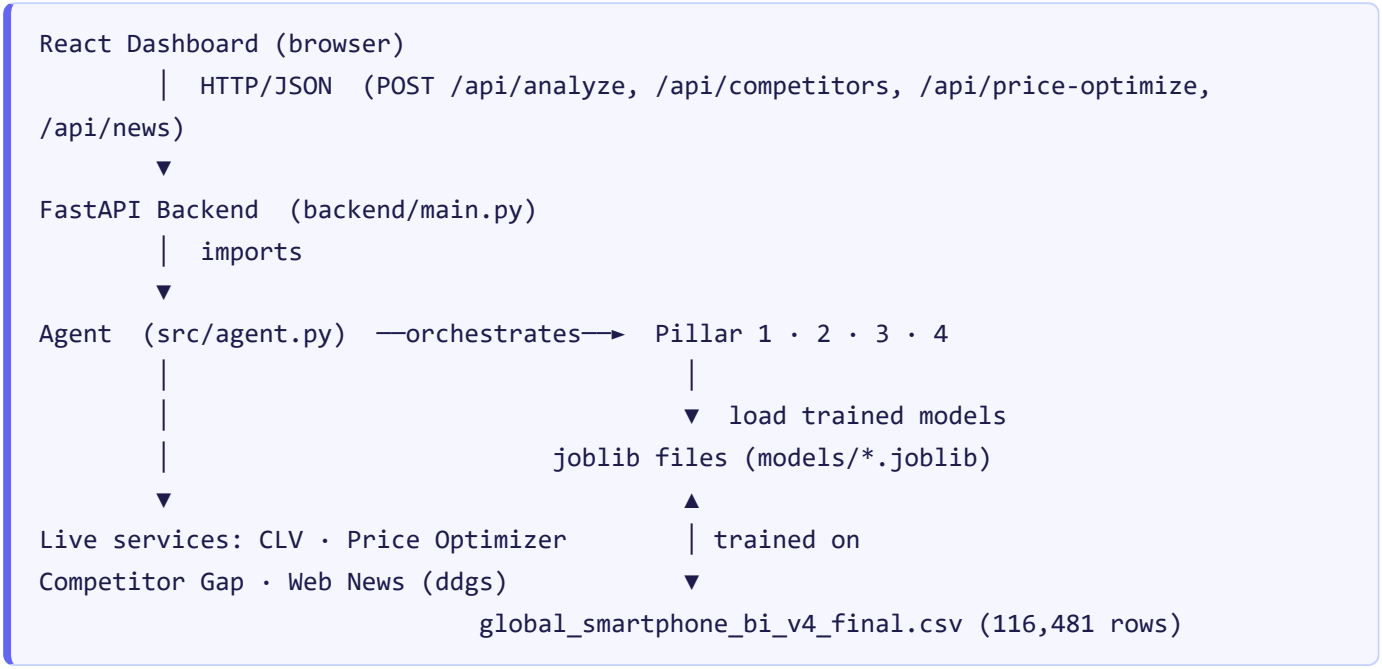
Table of Contents

1. System Overview & Architecture
2. Technology Stack (every layer & why)
3. The Dataset
4. The Feature Contract & Engineered Features (with equations)
5. Pillar 1 — Market Research (Segment Classifier + Similarity Engine)
6. Pillar 2 — Demand Forecasting
7. Pillar 3 — Customer Segmentation (Retrieval)
8. Pillar 4 — Product-Market Fit & Failure Prediction
9. Live Computations — CLV, Price Optimizer, Competitor Gap, Market Intelligence
10. Backend (FastAPI) — API Reference
11. Frontend (Next.js) — Architecture
12. Request Lifecycle — how everything connects
13. Accuracy Scorecard & Honest Limitations
14. Mathematical Appendix — worked examples

1. System Overview & Architecture

The Smartphone BI Agent takes a **single input** — the specifications of a new (unreleased) phone — and produces a full launch-intelligence dashboard. It is built as **four independent machine-learning pillars** plus three live analytical services, unified behind one agent and exposed through a REST API to a React dashboard.

1.1 High-level data flow



1.2 The four ML pillars

Pillar	Question it answers	Technique	Output
1. Market Research	What class of phone is this & what does it compete with?	XGBoost classifier + KNN similarity	Segment + nearest phones
2. Demand Forecast	How many units/month will it sell?	XGBoost regression (cold-start)	6-month unit curve
3. Customer Segmentation	Who will buy it?	KNN retrieval of real buyers	Age / income / gender / loyalty mix
4. Product-Market Fit	Will it succeed or fail?	Random-Forest classifier	PMF score + GO / CAUTION / RECONSIDER

1.3 Design principle — honesty over fake accuracy

Every cold-start model is allowed to see **only launch-known features** (specs, price, brand, region, months-since-launch). Post-launch signals (sales, social-media mentions, returns) are forbidden as inputs

— they would leak the answer. Train/test splits are done **by phone** (a phone never appears in both folds) so memorisation cannot inflate scores.

2. Technology Stack

2.1 Machine Learning & Data layer (Python)

Technology	Role	Why chosen
pandas	Load & manipulate the 116K-row dataset	Standard tabular data tool
NumPy	Vector math, log transforms, distance weights	Fast numerical core
XGBoost	Segment classifier (P1) & demand regressor (P2)	Best-in-class for tabular data; handles mixed features
scikit-learn	KNN (P1/P3), Random Forest (P4), scalers, encoders, group split, metrics	Reliable, well-tested ML toolkit
joblib	Serialise trained models to `models/*.joblib`	Fast load at inference

2.2 Backend layer

Technology	Role
FastAPI	REST API server exposing the agent & live services
Uvicorn	ASGI server that runs FastAPI
Pydantic v2	Validates & coerces the incoming phone-spec JSON
ddgs (DuckDuckGo Search)	Fetches live news / reviews / competitor articles
ThreadPoolExecutor	Runs all web searches in parallel (~4.5s vs ~15s)

2.3 Frontend layer

Technology	Role
Next.js 16 (React 19)	App-router SPA, client components
TypeScript	Typed API contracts between UI and backend
Tailwind CSS v4	Utility-first styling, dark/light theming
shadcn/ui	Accessible Card / Badge / Slider / Select components
Recharts	Demand bars, donuts, treemap, radar charts
Remotion	Programmatic explainer / demo videos

3. The Dataset

File: `global_smartphone_bi_v4_final.csv`

116,481
rows (phone × month × region)

66
columns

968
unique phones

6
regions ·
46
brands

Each row is one snapshot of a phone in a region for a month. Market segments are budget mid-range premium flagship.

Column families: identity (brand, model, country), numeric specs, categorical specs, time/region context, post-launch outcomes (sales, returns, NPS …), and buyer demographics (age, income, gender, RFM loyalty — each stored as percentage distributions).

4. The Feature Contract & Engineered Features

The single most important design artefact (`src/config.py`): every column is classified as **known-at-launch** or **post-launch**. Cold-start models may use only the former.

4.1 Known-at-launch inputs (29)

Numeric specs: price_usd, 5g_or_not, num_cores, processor_speed, battery_capacity, fast_charging_available, fast_charging, ram_capacity, internal_memory, screen_size, refresh_rate, num_rear_cameras, primary_camera_rear, primary_camera_front, extended_memory_available, resolution_height, resolution_width, avg_rating.
Categorical: brand_name, processor_brand, os. **Context:** months_since_launch, month_index, region.

4.2 Post-launch (NEVER used as cold-start inputs)

units_sold_monthly, social_media_mentions, search_trend_score, news_sentiment_score, market_share_pct, return_rate_pct, warranty_claims_per_100, nps_score, recommendation_rate_pct, repeat_purchase_rate_pct, customer_lifetime_value_usd, …

4.3 Engineered features (derived only from launch specs)

```
perf_score        = num_cores × processor_speed
cam_total         = primary_camera_rear + primary_camera_front
pixels            = resolution_height × resolution_width
battery_per_inch  = battery_capacity / screen_size
ram_x_mem         = ram_capacity × internal_memory
price_per_ram     = price_usd / (ram_capacity + 1)
value_score       = (ram_capacity + battery_capacity/1000 + cam_total/10) / ln(1 +
price_usd)
```

Example (Galaxy Test A: 8 cores @ 2.4 GHz, 50+16 MP, 5000 mAh, 6.5", 8 GB+128 GB, $320):
perf_score = 8×2.4 = **19.2** · cam_total = 66 · battery_per_inch = 5000/6.5 = **769.2** · price_per_ram = 320/9 = **35.6** · value_score = (8 + 5 + 6.6) / ln(321) = 19.6 / 5.77 = **3.40**

5. Pillar 1 — Market Research

Two jobs in one module (`src/pillar1_market_research.py`).

5.1 (A) Segment Classifier — XGBoost

Predicts `market_segment ∈ {budget, mid-range, premium, flagship}` from specs.

Setting	Value
Algorithm	XGBoost Classifier (gradient-boosted trees)
Hyper-parameters	n_estimators=300, max_depth=6, learning_rate=0.1, subsample=0.9
Features	18 numeric specs + 3 categorical (ordinal-encoded)
Split	GroupShuffleSplit by phone, 80/20
Result	Accuracy **99.4%**, Macro-F1 **99.6%**

Gradient boosting builds trees additively; the prediction is

```
ŷ = softmax( Σ_k  f_k(x) ),   f_k ∈ trees,   minimising  Σ loss(y, ŷ) + Σ Ω(f_k)
```

where Ω penalises tree complexity. Segments are almost perfectly separable by price+specs, so near-perfect accuracy here is legitimate (not leakage).

5.2 (B) Similarity / Retrieval Engine — KNN

Finds the most spec-similar phones that actually launched. Powers competitors, demographics (Pillar 3) and price anchoring.

```
1.  log-scale price:  price' = ln(1 + price_usd)
2.  standardise each feature:  z_i = (x_i − μ_i) / σ_i
3.  weight price 2.5× (price dominates phone "type"):  z_price ×= 2.5
4.  distance(a,b) = √ Σ_i (z_a,i − z_b,i)²      (Euclidean, region-filtered)
5.  return the k nearest neighbours
```

Reference catalogue: **5,801** phone×region rows.

6. Pillar 2 — Demand Forecasting

`src/pillar2_demand.py` — predicts `units_sold_monthly` for a brand-new phone using only launch-known features.

6.1 Target transform

Demand is right-skewed, so the model learns the log of demand:

```
target  y = ln(1 + units_sold_monthly)        (log1p)
prediction back-transform:  units = e^ŷ − 1   (expm1)
```

6.2 Leak-free brand prior

A brand-strength feature is computed on the **training fold only** and mapped onto both folds, so a test phone's own sales never enter its features:

```
brand_demand_prior(b) = mean over train rows of brand b of  ln(1 + units)
(unseen brands fall back to the global train mean)
```

6.3 Model & result

Setting	Value
Algorithm	XGBoost Regressor
Hyper-parameters	n_estimators=700, max_depth=8, learning_rate=0.03, subsample=0.85, colsample_bytree=0.8, min_child_weight=3
Features	specs + engineered + time + region + brand_demand_prior
Result	R^2(log) = **0.854**, MAE = **±9.07 units/month**

The 6-month curve is produced by sweeping `months_since_launch = 0…5` and back-transforming each prediction.

> **Honest ceiling.** $R^2 \approx 0.85$ is the genuine limit of what specs explain about demand; the remaining ~15% is marketing, timing and luck that no spec-only model can recover.

7. Pillar 3 — Customer Segmentation (Retrieval)

`src/pillar3_segmentation.py`. Phone specs do **not** directly predict who buys a phone (a direct regressor scores $R^2 \approx 0$). Instead we **retrieve** the real buyers of similar phones.

```
1. find the k=10 nearest phones in the same region (Pillar-1 engine)
2. average their real buyer distributions (age, income, gender, RFM)
3. clip negatives, renormalise each block to sum to 100%:
       p_i = max(0, v_i) / Σ_j max(0, v_j) × 100
```

Block	R^2	Verdict
Income distribution	**0.771**	Strong — similar phones share buyer income
Age distribution	**0.671**	Good — region-awareness doubled it
RFM / loyalty	**−0.16**	Not predictable from specs (reported honestly)

Why loyalty is shown but flagged. Loyalty depends on the person & brand relationship, not the device. The dashboard therefore presents loyalty as a population-level prior, not a confident prediction.

8. Pillar 4 — Product-Market Fit & Failure Prediction

`src/pillar4_pmf.py`. The dataset has no "success" column, so a transparent PMF target is **constructed** from real post-launch outcomes, then a classifier learns to predict it from launch specs only (non-circular: none of the label's ingredients are specs).

8.1 PMF score formula (each component min-max normalised to 0–1)

```
PMF = 0.25 · NPS
    + 0.20 · repeat_purchase_rate
    + 0.20 · recommendation_rate
    + 0.15 · (1 − return_rate)        ← inverted (low returns = good)
    + 0.10 · (1 − warranty_claims)    ← inverted (few defects = good)
    + 0.10 · sales_slope              ← are monthly sales trending up?

where  minmax(x) = (x − min) / (max − min + ε)
```

Weights: loyalty/advocacy 65%, quality 25%, momentum 10%.

8.2 Failure label & classifier

```
failure = 1  if  PMF < 25th-percentile(PMF)   else 0      (data-relative, balanced)
```

Setting	Value
Algorithm	Random Forest Classifier
Hyper-parameters	n_estimators=500, max_depth=12, min_samples_leaf=3, class_weight="balanced"
Result	Accuracy **79.9%**, AUC **0.88**, base-rate 25%

8.3 Recommendation mapping (inference)

```
fail_p = P(failure | specs)          PMF_shown = 1 − fail_p
fail_p < 0.15  → "GO FOR LAUNCH"
fail_p < 0.35  → "PROCEED WITH CAUTION"
else           → "RECONSIDER / REWORK"
```

Top failure drivers (feature importance): price, avg_rating, screen resolution, 5G, RAM.

9. Live Computations

9.1 Customer Lifetime Value (computed live in the agent)

```
repeat_factor   = 1 + repeat_rate / 100
net_price       = price × (1 − return_rate / 100)
loyalty_horizon = 2.4          (avg purchase cycles a satisfied buyer gives a brand)
margin          = 0.18
CLV = net_price × margin × repeat_factor × loyalty_horizon
```

Worked example (Galaxy Test A, $320, loyal-rate 25.9%, return 8%):
repeat_factor = 1.259 · net_price = 320×0.92 = 294.4
CLV = 294.4 × 0.18 × 1.259 × 2.4 = **$160.1** ✓ (matches dashboard)

9.2 Optimal Price Recommender (anchored to competitors → stable)

```
market_price = median( prices of the 5 spec-matched competitors )   ← price excluded from matching
spec_ratio   = clip( your_spec_score / competitor_avg_spec , 0.8 , 1.25 )
fair_price   = market_price × spec_ratio
unit_cost    = fair_price × (1 − 0.40)              (40% gross-margin assumption)
demand(p)    = base_demand × max(0, 1 − 1.3 · (p − fair_price)/fair_price)   (linear elasticity)
revenue(p)   = p × demand(p)
profit(p)    = (p − unit_cost) × demand(p)
recommended  = argmax_p  profit(p) × PMF(p)
```

Why it is stable. Competitors are matched on **specs only** (price excluded), so editing your input price never moves the anchor — the recommendation converges instead of chasing the input upward.

Example (specs of Galaxy Test A): competitors $301–$361, median $325 → fair ≈ $326, unit_cost ≈ $196, recommended ≈ **$383** (peak profit×PMF).

9.3 Competitor Gap Analysis

```
norm(value) = clip( (value − lo) / (hi − lo) × 100 , 0 , 100 )
```

Six specs (RAM, battery, camera, screen, refresh, performance) are normalised 0–100 for the phone and its three nearest competitors and drawn as a treemap; differences ≥12% become plain-English win/loss insights.

9.4 Market Intelligence (live web)

Two buckets, searched in parallel via DuckDuckGo: **Your Phone** (model + spec-similar handsets) and **Competitor** (the rivals Pillar 1 predicts). Results are de-duplicated, slot-capped and round-robin interleaved so neither bucket dominates.

10. Backend — API Reference (FastAPI)

Endpoint	Method	Purpose	Returns
`/api/health`	GET	Liveness check	status
`/api/analyze`	POST	Full pipeline (4 pillars + CLV)	segment, demand, profile, PMF, CLV
`/api/competitors`	POST	Spec gap vs nearest rivals	radar scores, specs, insights
`/api/price-optimize`	POST	Profit-max price curve	curve, recommended price, assumptions
`/api/news`	POST	Live news / reviews	articles (Your Phone / Competitor)

Input is validated by a Pydantic `PhoneSpec` model (with the `5g_or_not` alias). CORS is open to `localhost:3000`. The agent & pillars are imported from `src/`; trained models are loaded from `models/*.joblib` on demand.

11. Frontend — Architecture (Next.js)

File	Responsibility
`components/bi-agent-app.tsx`	Root client app: state, theme, welcome orb, loading screen, parallel fetches
`components/specs-form.tsx`	Sidebar spec inputs (chips, steppers, selects, toggles)
`components/dashboard.tsx`	Assembles all result sections
`components/charts/*`	Demand bars, donuts, loyalty bar, competitor treemap, price radar, PMF gauge
`components/news-panel.tsx`	Live news cards + filters
`lib/api.ts` / `lib/types.ts`	Typed fetch helpers & response contracts

On **Analyze**: the analysis call blocks the dashboard render; news, competitors and price optimisation are fired in parallel and populate their sections as they resolve, so the core dashboard appears in ~2s while slower web search fills in afterward.

12. Request Lifecycle — end to end

```
1. User fills specs → clicks Analyze
2. React builds a PhoneSpec JSON, POSTs to 4 endpoints (analyze blocks; others
background)
3. FastAPI validates with Pydantic, calls agent.analyze()
4. Agent runs:
     P1 predict_segment()         → XGBoost classifier (joblib)
     P1 find_similar()            → KNN over scaled catalogue
     P3 predict_demographics()    → average neighbours' real buyers
     P2 predict_curve()           → XGBoost regressor, 6 months
     P4 predict_fit()             → Random-Forest probability
     estimate_clv()               → closed-form CLV
5. JSON returned → React renders KPIs, charts, profile, PMF
6. Background: /competitors (treemap), /price-optimize (radar), /news (cards) fill in
7. Dashboard complete
```

13. Accuracy Scorecard & Honest Limitations

Pillar / Model	Metric	Score
P1 Segment (XGBoost)	Accuracy / Macro-F1	**99.4% / 99.6%**
P2 Demand (XGBoost)	R^2(log) / MAE	**0.854 / ±9.07 units**
P3 Income (KNN)	R^2	**0.771**
P3 Age (KNN)	R^2	**0.671**
P3 RFM loyalty (KNN)	R^2	**−0.16 (not predictable)**
P4 Failure (Random Forest)	Accuracy / AUC	**79.9% / 0.88**

Honest limitations

- Demand caps at $R^2 \approx 0.85$ — the data's true ceiling from specs alone.
- RFM/loyalty is not predictable from specs; shown as a population prior.
- The PMF label is constructed (transparent formula), not ground-truth.
- Price-optimizer cost & elasticity are stated assumptions (40% margin, $\beta=1.3$).
- Truly novel form-factors have weak nearest neighbours.

14. Mathematical Appendix — Worked Examples

A. Segment confidence

```
confidence = max_k softmax(score_k);  Galaxy Test A → mid-range @ 100%
```

B. Demand back-transform

```
model outputs ŷ=4.66 (log) → units = e^4.66 − 1 ≈ 105 units/month
```

C. Price-vs-category

```
price_delta = (price − category_avg)/category_avg × 100
($320 vs mid-range avg $247) → +29.5%
```

D. Affordability fit (frontend)

```
comfortable% = Σ income-tier% where price ≤ tier_budget_ceiling
tiers: Low ≤ $180, Lower-mid ≤ $350, Mid ≤ $650, Upper-mid ≤ $1100, High ≤ ∞
```

E. Loyalty split (frontend)

```
engaged% = (Champion + Loyal + Recent) / (engaged + at-risk) × 100
```

Generated from the live source code of the Smartphone BI Agent (src/config.py, pillar1–4, agent.py, backend/main.py, frontend/components/*). All metrics read from outputs/metrics.json.